

PRACTICE PROBLEMS – RECURSION, BACKTRACKING, AND N-QUEENS

PROBLEM 1: RECURSION – DIRECTORY SIZE CALCULATION

a) Recursive Function Pseudocode

Function getDirectorySize(path):

 If path is a file:

 Return file size <-- Base Case

totalSize = 0

For each item in directory:

 totalSize += getDirectorySize(item) <--
Recursive Case

Return totalSize

Base Case:

- When path is a file, return its size.

Recursive Case:

- When path is a directory, recursively calculate sizes of all contents.

b) Execution Trace (Call Stack and Returns)

Directory Structure Sizes:

main.java = 100 KB

utils.java = 50 KB

readme.txt = 10 KB

setup.pdf = 200 KB

config.xml = 20 KB

Execution Flow:

getSize(project)

├─ getSize(src)

| ├─ getSize(main.java) → 100

| └─ getSize(utils.java) → 50

| src returns → 150

|

├─ getSize(docs)

| ├─ getSize(readme.txt) → 10

| └─ getSize(guides)

| └─ getSize(setup.pdf) → 200

| guides returns → 200

| docs returns → 210

|

└─ getSize(config.xml) → 20

Final Return:

project = 150 + 210 + 20 = 380 KB

c) Time Complexity

Time Complexity = $O(N)$

Where N = total number of files + directories

Reason:

Each file or directory is visited exactly once.

d) Iterative Solution

Yes, can be solved iteratively using:

Data Structure: Stack or Queue

Iterative Idea:

Push root directory into stack.

While stack not empty:

- Pop item

- If file → add size

- If directory → push children

Comparison:

Recursive:

- Cleaner and easier to implement
- Uses system call stack

Iterative:

- Avoids recursion depth limits
- More manual control

e) Handling Symbolic Link Cycles

Problem:

Symbolic links can create infinite loops.

Solution:

Maintain a visited set of directory paths.

Modified Base Case:

If path already visited → Return 0

=====

=====

PROBLEM 2: BACKTRACKING – WORD SEARCH

=====

=====

a) Backtracking Algorithm Pseudocode

State Representation:

- Current position (row, col)
- Current index in word
- Visited cells

Function search(r, c, index):

 If index == word length:

 Return True <-- Goal Condition

 If out of bounds OR already visited OR
 grid[r][c] != word[index]:

 Return False <-- Constraint Check

 Mark visited[r][c] = True

 For all 8 directions:

 If search(next cell, index + 1):

 Return True

 visited[r][c] = False <-- Backtracking

 Return False

b) Decision Tree (Example Path)

Start $D \rightarrow E \rightarrow R \rightarrow E \rightarrow A \rightarrow M$

Successful Path Example:

$D(2,0) \rightarrow E(2,1) \rightarrow A(2,2) \rightarrow M(2,3)$

Backtracking happens when next character does not match.

c) Execution Trace

Step 1: Find D

Step 2: Try all neighbors for next letter

Step 3: If mismatch → backtrack

Step 4: Restore visited state

Step 5: Try next option

d) Worst Case Time Complexity

$$O(N \times M \times 8^L)$$

Where:

$N \times M$ = Grid size

L = Word length

8 = Max branching directions

e) Finding ALL Possible Ways

Modification:

Do not return immediately when found.

Store each valid path.

Additional Data Structure:

List of paths

=====

=====

PROBLEM 3: N-QUEENS OPTIMIZATION

=====

=====

a) Search Space Sizes

Brute Force:

Each queen can be in any square $\rightarrow N^N$

For $N=8 \rightarrow 64^8$ possibilities

One Queen Per Row:

Now each row has N choices $\rightarrow N^N$

For $N=8 \rightarrow 8^8 = 16,777,216$

Actual Backtracking Exploration:

Much smaller due to pruning ($\sim$ thousands).

b) Optimized N-Queens Pseudocode

Use:

board[row] = column position

colUsed[]

diag1Used[row - col + N]

diag2Used[row + col]

Function placeQueen(row):

 If row == N:

 Save solution

 Return

For col in 0 to N-1:

 If column and diagonals free:

 Place queen

 Mark arrays

 placeQueen(row + 1)

Remove queen

Unmark arrays

c) Trace for $N = 4$

Board Representation Example:

[1, 3, 0, 2]

Backtracking occurs when no valid column found in row.

d) Finding Only One Solution Fast

Modification:

Return True immediately when first solution found.

Stop further recursion.

e) N-Queens with Forbidden Squares

Modification:

Add check:

If square is forbidden → skip

Complexity Impact:

Search space reduces but still exponential worst case.